Fetching Data From an API

Contents

1. Overview . . . 2
2. What an API is . . . 3
3. Why APIs exist: one database, many apps . . . 3
    3.1 Railway tickets . . . 3
    3.2 One company, many platforms . . . 3
4. Asking an API for data . . . 4
    4.1 The API key . . . 5
5. Reading the reply . . . 5
    5.1 Planning the dataset . . . 6
6. Fetching one page in Python . . . 6
7. Looping over every page . . . 8
8. Saving the dataset . . . 10
9. Sharing datasets and finding more APIs . . . 11
10. Summary . . . 12
11. Sources . . . 12
12. Key terms . . . 13

Where this fits: Pipeline map step 2 (Get data). See the Course map.

0 Foundations	1 Frame	**2 Get data**	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Deployment; JSON and SQL data.
- **Compare with:** Web scraping.

1. Overview

Key point: We can build our own dataset by asking a website's API for its data, page by page, and joining the replies into one DataFrame.

CSV files (reading a CSV file: plain text, one row per line), JSON files (what JSON is: text made of named values and lists) and SQL databases (what SQL is: tables we query) hold data that someone has already collected. Often the data we want is still on a website, and no file exists. Many websites offer an **API** (G-204) that hands out their data on **request** (G-1674), so we can collect it ourselves.

Figure 1 shows the whole process. We send a request, get back a reply in JSON, turn it into a DataFrame (a pandas table of rows and columns), repeat for every page, join the pages and save the result as a CSV file. Follow the arrows: along the top row from left to right, down, then along the bottom row from right to left; the dashed red arrow loops back to the server for every page.

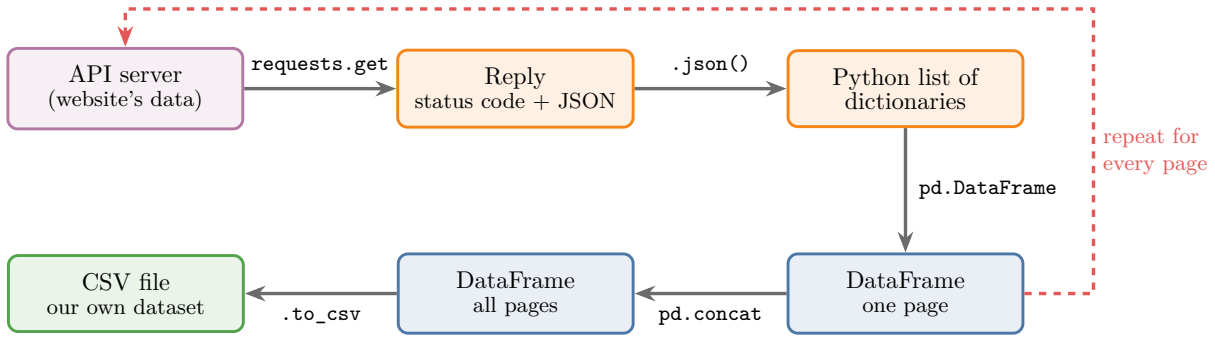


Figure 1: From an API reply to our own CSV dataset

The Notebook for this Note (`ML-016-fetching-data-from-api.ipynb`) runs every step, with saved copies of the data in `data/`.

2. What an API is

> **Key point:** An **API** (G-204) lets two pieces of software talk to each other; it carries data from one point to another.

API stands for **Application Programming Interface**. Almost everything we do today runs through some app, and behind the apps, APIs are what let different programs work together.

The simplest definition: an API lets two pieces of software talk to each other. Another way to see it: an API is a **data pipeline**, a channel that carries information from one point to another.

3. Why APIs exist: one database, many apps

> **Key point:** A company wraps its database in an **API** (G-204), so other apps and platforms can use the same data without ever getting the database itself.

3.1 Railway tickets

A train seat in India can be booked through many apps: IRCTC's own website, MakeMyTrip, Yatra and others. These are different companies, each with its own database. Yet if we book a seat through IRCTC, a person trying to book the same seat on MakeMyTrip sees it as booked.

IRCTC does not hand its database to MakeMyTrip. Instead, IRCTC writes functions around its database: the **API** (G-204) (Figure 2a). When MakeMyTrip calls one of these functions with the right input, such as the train number, the date and the seat, IRCTC's server replies with the answer, in JSON: is the seat free or not?

3.2 One company, many platforms

Facebook has a website, a mobile website, an Android app and an iPhone app. All of them work on a single database, through one **API** (G-204) (Figure 2b). The shared API is why a post written on the phone appears on the website at once.

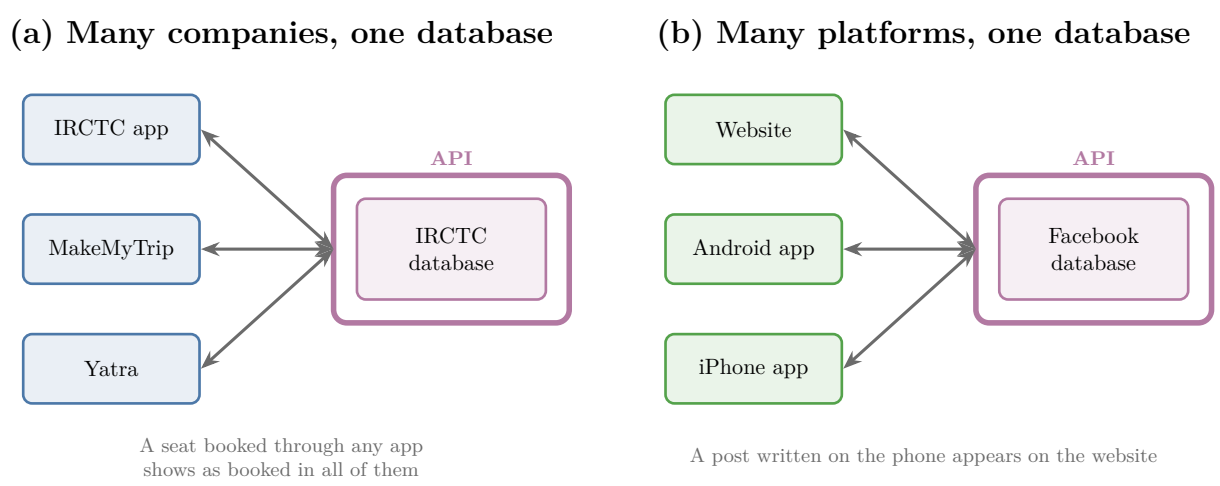


Figure 2: An API in front of a database serves many apps

> **Extra:** The app that asks is called the **client**, and the computer that answers is the **server**. Building APIs (in Python, for example with Flask or FastAPI) is a skill of its own; in this Note we only use them.

4. Asking an API for data

> **Key point:** A **request** (G-1674) to an API is a web address (URL): which data we want, who we are (the API key), and which page.

Our example website is **TMDB** (The Movie Database, `themoviedb.org`), a large database of movies. Its developer pages (`developers.themoviedb.org`) list many **endpoints**: addresses that each return one kind of data. One of them, `/movie/top_rated`, returns the top-rated movies in TMDB's database.

The three parts are glued into one text, as in this example URL (the key is a placeholder):

```
https://api.themoviedb.org/3/movie/top_rated?api_key=MYKEY&page=1
```

The `?` marks the end of the address and the start of the settings; each setting is a name, an equals sign and a value; a second setting is joined to the first by `&`. Figure 3 takes the request apart:

- **Base address:** where the **API** (G-204) lives.
- **Endpoint:** what we want, here the top-rated movies.
- **Query parameters:** settings after the `?`, joined by `&`. Here the **API key** (G-205) and the page number.

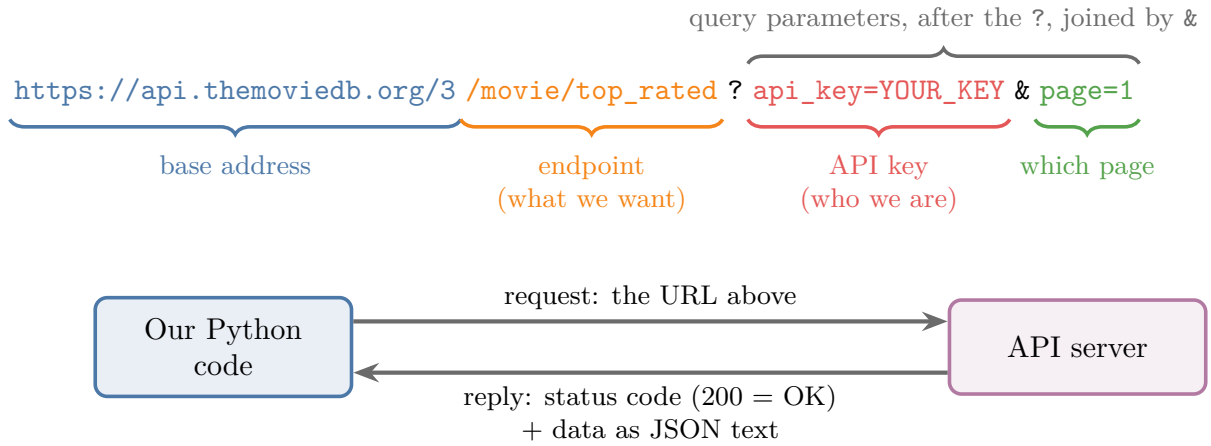


Figure 3: The parts of an API request, and the reply it brings back

4.1 The API key

An **API key** (G-205) is a secret code that tells the **API** (G-204) who is asking. To get a TMDB key, we create a free account, log in, and open Settings, then the API section.

Each key has a limit on how many **requests** (G-1674) it can make. So every user should create their own key and never share it: if many people use one key, it runs out.

> **Extra:** Keep keys out of code. A key pasted into a notebook gets shared with the notebook, for example on GitHub. A safer way is to store it in an **environment variable**, a named value that lives on our computer, and read it in Python with `os.environ["TMDB_API_KEY"]`. The Python box in Section 6 does this.

5. Reading the reply

> **Key point:** The reply is JSON: a dictionary whose `results` key holds a list of dictionaries, one per movie, plus the page count we need for the loop.

If we paste the **request** (G-1674) URL into a web browser, the reply appears as a long block of text. That text is JSON (see what JSON is). A **JSON viewer**, such as `jsonviewer.stack.hu`, lays the text out as a tree so we can see its structure.

A **dictionary** (G-603) is a set of name and value pairs, written `{"page": 1, "total_pages": 428}`; a looked-up name gives its value. A **list** is an ordered row of items, written `["a", "b", "c"]`, and each item has a position starting from 0. Figure 4 shows the structure. JSON looks just like a Python dictionary:

- `page`: which page this is, here 1.
- `results`: a list. Inside it, one dictionary per movie, 20 movies per page. The first top-rated movie is *Dilwale Dulhania Le Jayenge*.
- `total_pages`: 428 pages in all.

- `total_results`: 8,551 movies in all.

Reply to /movie/top_rated, page 1

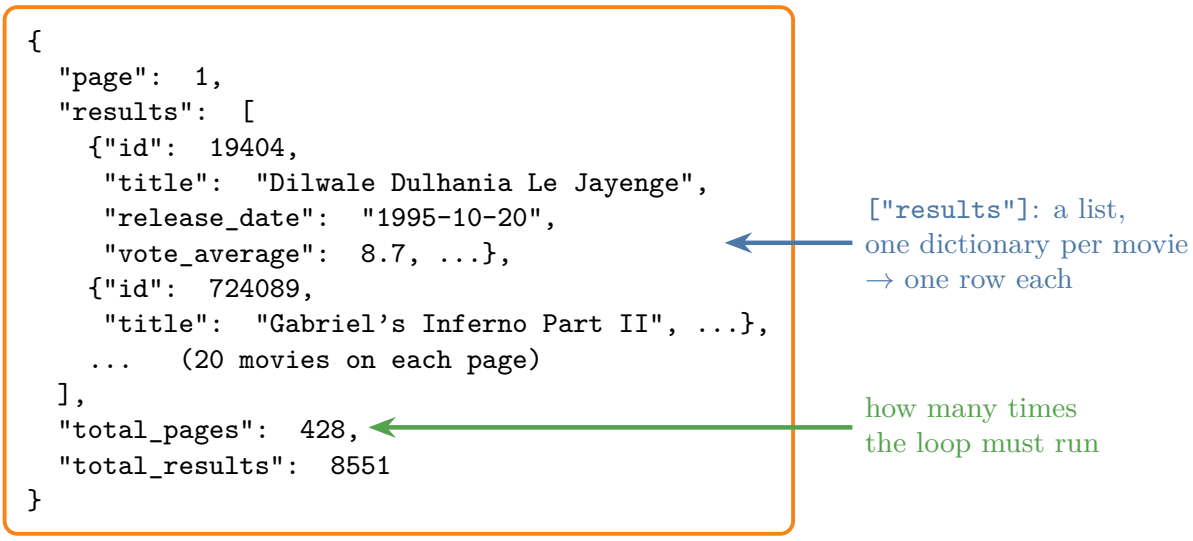


Figure 4: The structure of one reply from the TMDB API

5.1 Planning the dataset

Before writing code, we decide what the final table should hold. Each movie will be one **observation** (one record, one row of the table). Each value we keep will be a **feature** (a variable describing the movie, one column of the table). From each movie's **dictionary** (G-603) we keep 7 columns: `id`, `title`, `overview` (a short plot summary), `release_date`, `popularity`, `vote_average` and `vote_count`.

The plan follows from the reply's structure:

1. **Request** (G-1674) one page and turn its `results` into a DataFrame of 20 rows.
2. Do this 428 times, once per page, adding each page's rows to one big DataFrame.
3. The final table should have shape (8551, 7).

6. Fetching one page in Python

> **Key point:** `requests.get(url)` sends the request; `.json()` turns the reply into Python objects; `pd.DataFrame` turns a list of dictionaries into a table.

We need two libraries: pandas, and **requests** (G-1674), which sends web requests from Python. Both come with Anaconda.

`requests.get(url)` sends the request and returns a **response** (G-1686) object. The response carries a **status code** (G-1886), a number that says how the request went (RFC 9110, §15):

Status code	Meaning
200	OK: everything worked
401	Not authorised: missing or wrong API key
404	Not found: what we asked for does not exist
500	Server error: the server is down or broken

`response.json()` reads the JSON text into Python dictionaries and lists. Then `["results"]` picks out the list of movies, and `pd.DataFrame(...)` makes it a table: one row per **dictionary** (G-603), one column per key. Finally, a list of column names keeps only the 7 columns we planned.

Python: One page from TMDB (needs a free key, stored in an environment variable).

```
import os
import pandas as pd
import requests

url = "https://api.themoviedb.org/3/movie/top_rated"
settings = {"api_key": os.environ["TMDB_API_KEY"],
            "language": "en-US", "page": 1}
response = requests.get(url, params=settings)
response.status_code          # 200

movies = response.json()["results"]   # list of 20 dicts
columns = ["id", "title", "overview", "release_date",
           "popularity", "vote_average", "vote_count"]
temp_df = pd.DataFrame(movies)[columns]
temp_df.shape                 # (20, 7)
```

`params=` builds the part after the `?` for us, so we never paste the key into the URL in our code. The request that goes out still carries the key in its address, as in Figure 3; only the code stays free of it.

Extra: What changed. TMDB still works, but only with a personal key. Without one it replies with status code 401 and the message "Invalid **API key** (G-205)". So the Notebook uses **TVmaze** (`api.tvmaze.com`), a free TV-show database that needs no key. TVmaze returns the same kind of data: shows with a name, premiere date, average rating, popularity score (`weight`) and summary. A saved real reply, `data/tvmaze_shows_page0.json`, lets the Notebook run without internet (checked October 2026).

The TVmaze reply differs from TMDB's in two ways:

- The reply is the list itself, with no `results` key around it, and no total page count.
- Each page holds up to 250 shows (TVmaze **API** (G-204) docs); page 0 has 240, because deleted shows leave gaps.

Python: One page from TVmaze (no key).

```
url = "https://api.tvmaze.com/shows?page=0"
response = requests.get(url, timeout=10)
shows = response.json()          # list of 240 dicts

page_df = pd.json_normalize(shows)
columns = ["id", "name", "language", "premiered",
           "rating.average", "weight", "summary"]
page_df = page_df[columns]
page_df.shape                    # (240, 7)
```

`timeout=10` gives up after 10 seconds instead of waiting forever.

Extra: In TVmaze, the rating is a small dictionary inside each show: `"rating": {"average": 6.6}`. `pd.DataFrame` would put the whole dictionary into one cell. `pd.json_normalize` (G-99) opens nested dictionaries into their own columns, so we get a column `rating.average` holding 6.6 (Figure 5).

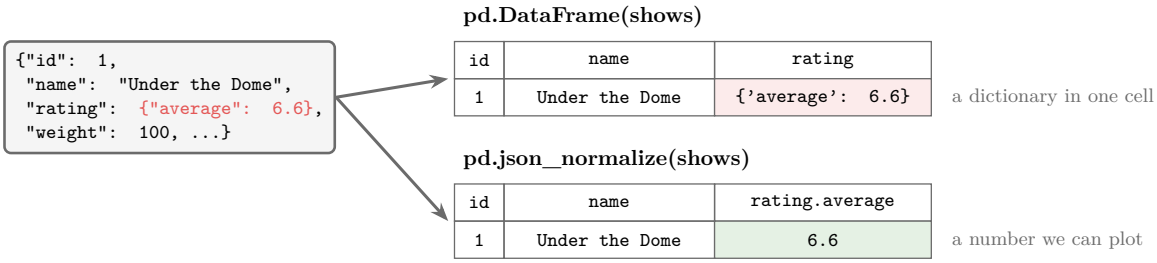


Figure 5: The first show of the saved TVmaze reply. `pd.DataFrame` keeps the nested rating dictionary in one cell; `pd.json_normalize` opens it into its own column, `rating.average`

7. Looping over every page

Key point: A loop repeats the one-page code for every page; `ignore_index=True` numbers the joined rows 0, 1, 2, ... without repeats.

One page is only a small part of the data. To get everything, we run the one-page code in a **for loop**, once per page, changing only the page number in the URL. For TMDB that is `range(1, 429)`: pages 1 to 428, because `range` stops just before its end value.

Each pass builds a small DataFrame, `temp_df`, and adds it to a list. After the loop, `pd.concat` joins all the pages into one DataFrame. Figure 6 shows this with the first 5 TVmaze pages: 1,208 shows.

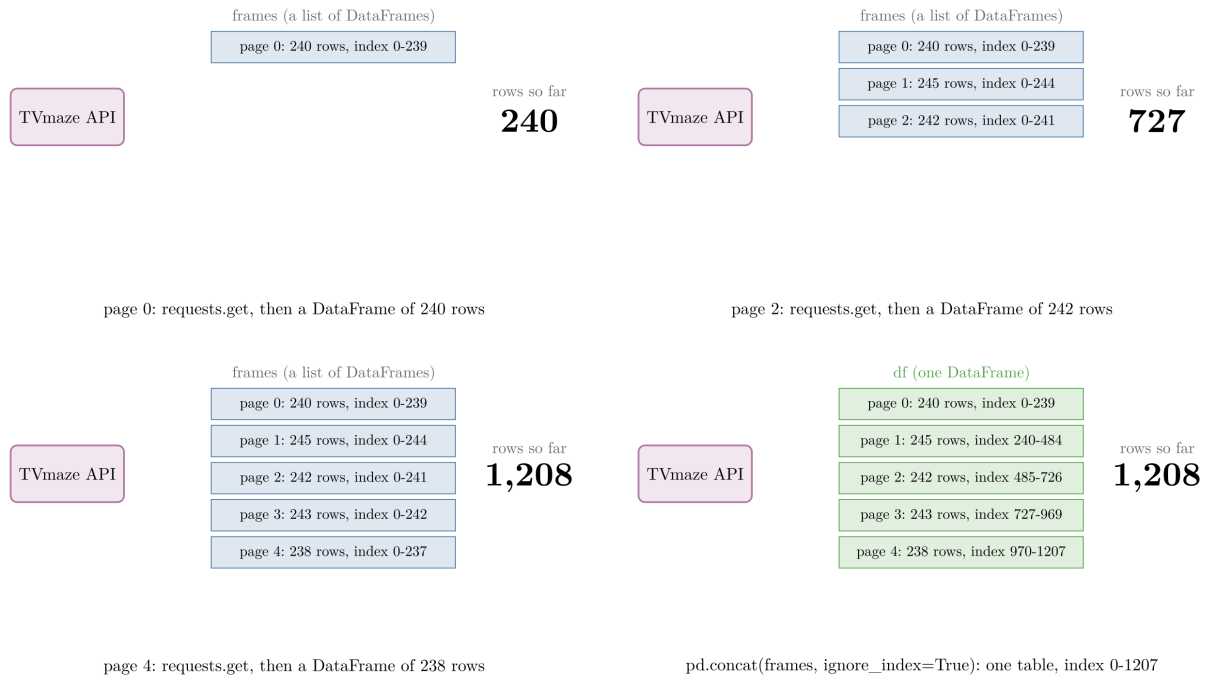


Figure 6: Five pages fetched one by one, then joined into one table

Every page's DataFrame numbers its rows from 0. Joined as they are, the index labels would repeat: 0 to 239, then 0 to 244 again, and so on. With `ignore_index=True`, pandas throws the old labels away and numbers the rows 0 to 1,207 (Figure 6, last frame).

Python: The loop (TVmaze, first 5 pages).

```
import time

frames = []                          # one DataFrame per page
for page in range(0, 5):
    url = f"https://api.tvmaze.com/shows?page={page}"
    response = requests.get(url, timeout=10)
    if response.status_code != 200:
        break                        # 404: no more pages
    temp_df = pd.json_normalize(response.json())[columns]
    frames.append(temp_df)
    time.sleep(0.5)                  # be gentle with the server

df = pd.concat(frames, ignore_index=True)
df.shape                             # (1208, 7)
```

An **f-string** such as `f"...{page}"` is text in which Python puts the value of `page` in place of `{page}`. `break` leaves the loop early. `frames.append(temp_df)` adds one item to the end of the list.

Fetching all of TMDB's 428 pages takes about two minutes, because each page is a separate

trip to the server. The result has 8,551 rows and 7 columns, as planned.

> **Extra:** Older code grew the table inside the loop with `df = df.append(temp_df, ignore_index=True)`. pandas 2.0 removed `DataFrame.append` (pandas release notes); it now fails with `AttributeError`. Collecting the pages in a list and calling `pd.concat` once at the end replaces it, and is also faster.

> **Extra:** Most **APIs** (G-204) have a **rate limit**: a maximum number of **requests** (G-1674) in a given time. TVmaze allows at least 20 requests every 10 seconds (TVmaze API docs) and answers status code 429 ("too many requests") beyond that. `time.sleep(0.5)` waits half a second between pages, which keeps us under the limit. TVmaze has about 380 pages (roughly 90,000 shows); `range(0, 400)` fetches them all, and the 404 after the last page ends the loop.

8. Saving the dataset

> **Key point:** `df.to_csv("file.csv")` saves the DataFrame as a CSV file, our own dataset ready for analysis.

Once the loop has finished, one line saves the table, for example as `movies.csv`. From then on we can load it with `read_csv` (see reading a CSV file) without calling the **API** (G-204) again.

> **Python:** Saving and reloading.
> ```
> df.to_csv("data/shows.csv", index=False)
> pd.read_csv("data/shows.csv").shape    # (1208, 7)
> ```
> `index=False` leaves out the row numbers. Without it, the file gets an extra unnamed first column holding 0, 1, 2, ...

Figure 7 is a first look at the dataset we built: most of the 1,056 rated shows score between 7 and 8.5 out of 10. The other 152 shows have no rating yet (missing values).

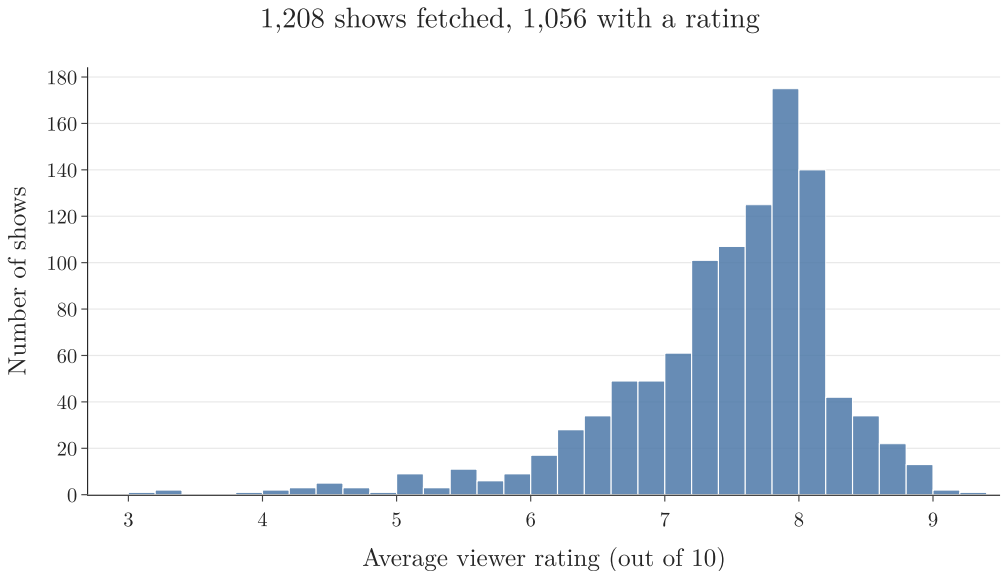


Figure 7: Average ratings of the 1,208 TV shows fetched from TVmaze

9. Sharing datasets and finding more APIs

> **Key point:** A dataset built from an **API** (G-204) can be shared on Kaggle, and RapidAPI lists many free APIs to build more.

Kaggle is a website where people share datasets, notebooks and write-ups, and take part in ML competitions. Uploading a dataset we built ourselves, such as the movies table, adds to our Kaggle profile. A strong profile (datasets, notebooks, competitions, discussions) helps when applying for internships and jobs.

For more data, **RapidAPI** (`rapidapi.com`) keeps a list of free APIs in many categories: football, cities, weather, real estate, Instagram, Amazon and more. Any of them works with the same recipe:

1. Read the API's documentation and find the endpoint.
2. Look at one reply in a JSON viewer and plan the columns.
3. Fetch one page into a DataFrame, then loop over all pages.
4. Save the result as a CSV file.

> **Extra:** Before sharing a dataset, check the API's terms of use. Some APIs allow personal use only, or ask that the data's source be credited. TMDB, for example, asks to be credited as the data source (TMDB API FAQ).

10. Summary

Step	Code	TMDB (original)	TVmaze (Notebook)
Request a page	`requests.get(url)`	needs an API key	no key
Read the reply	`response.json()`	dict; movies in `["results"]`	the list itself
One page to a table	`pd.DataFrame` / `pd.json_normalize`	20 rows	up to 250 rows
Loop over pages	`for page in range(...)`	428 pages	about 380 pages (5 in the Notebook)
Join the pages	`pd.concat(frames, ignore_index=True)`	8,551 rows, 7 columns	1,208 rows, 7 columns
Save	`df.to_csv(...)`	`movies.csv`	`data/shows.csv`

- An **API** (G-204) lets two programs talk; companies wrap a database in an API so many apps can share it.
- A **request** (G-1674) is a URL: base address, endpoint, and query parameters such as the API key and page, so changing only the page number fetches the next page.
- The **status code** (G-1886) says how it went: 200 OK, 401 bad key, 404 not found, 500 server error. So a failed request tells us what to fix: TMDB without a key answers 401.
- The reply is JSON; a list of dictionaries becomes a DataFrame with one row per **dictionary** (G-603). Nested dictionaries need `pd.json_normalize`, because `pd.DataFrame` would keep a whole dictionary in one cell.
- Loop over the pages, because one page holds only part of the data; collect the DataFrames in a list, then `pd.concat(..., ignore_index=True)`, because every page numbers its rows from 0 and the labels would repeat.
- Keep **API keys** (G-205) out of code, because a key in a notebook gets shared with the notebook and each key has a request limit; respect the API's rate limit, because beyond it the API answers 429.
- Save the joined table with `to_csv`, so the dataset we built loads with `read_csv` without calling the API again.

11. Sources

Built from

- CampusX, "Fetching Data From an API | Day 17 | 100 Days of Machine Learning", YouTube, https://www.youtube.com/watch?v=roTZJaxjnJc

Other references

- Fielding, R., Nottingham, M. and Reschke, J. (2022). HTTP Semantics. RFC 9110, IETF. Section 15: Status Codes. rfc-editor.org/rfc/rfc9110.
- pandas release notes. What's new in 2.0.0. pandas.pydata.org/docs/whatsnew.
- TMDB. API FAQ (attribution). developer.themoviedb.org/docs/faq.
- TVmaze. API documentation (show index, rate limiting). tvmaze.com/api.

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
requests (G-1674)	Python library that sends web requests.
API key (G-205)	A secret code that tells the API who is asking.
Dictionary (Python) (G-603)	A lookup table from keys to values, written `{key: value}`.
Response (G-1686)	What `requests.get` returns: the status code plus the reply.
Status code (G-1886)	A number saying how a request went: 200 OK, 401, 404, 500.
`json_normalize` (G-99)	pandas function that turns nested JSON into flat columns.
`ignore_index` (G-93)	Setting of `pd.concat` that renumbers the joined rows from 0.
Client, server (G-397)	The program that asks, and the computer that answers.
Data pipeline (G-538)	A channel that carries data from one place to another, such as an API that passes a company's data to our program.
Endpoint (G-688)	One address of an API that returns one kind of data.
Environment variable (G-693)	A named value stored on the computer outside the code and read with `os.environ`; used to keep secrets such as API keys out of notebooks that get shared.
f-string (G-741)	Text starting with `f` in which `{name}` is replaced by a value.
JSON viewer (G-989)	A tool that lays out JSON text as a tree to show its structure.
Query parameters (G-1604)	Settings written after the `?` in an API's URL and joined by `&`, such as `page=1` or the API key; they tell the API which data to return.
RapidAPI (G-1632)	A website listing many APIs, including free ones.
Rate limit (G-1634)	The most requests an API accepts in a given time.
API (Application Programming Interface) (G-204)	A service that returns data when our code asks for it; a website's API hands out its data on request.

Term	Meaning
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
`pd.concat` (G-125)	pandas function that joins several DataFrames into one.
Kaggle (G-999)	A website for sharing datasets and notebooks and for ML competitions.